

AGENT 1

COT:

1. Structural Analysis:

Steps:

- 1: Identify all major code blocks — functions, classes, control flows, and assignments.
- 2: For each block, note its exact start and end lines, and extract minimal code snippets.
- 3: Determine “contains” relationships (nested blocks within others).
- 4: Identify “control_flow”, “call”, or “data_flow” links between them.
- 5: Finally, output the JSON in the specified schema, without any explanation — just valid JSON.

Code with line numbers:

{lines}

2. data flow and control flow

Steps:

Analyze the code step-by-step as follows:

1. Extract individual code blocks (functions, conditionals, assignments).
2. For each block, specify what data it reads or writes.
3. Identify how control flows between consecutive blocks inside a function.
4. Generate “data_flow”, “control_flow”, and “contains” edges based on those relationships.
5. Output a single JSON object as per the schema provided earlier.

Code with line numbers:

{lines}

3. compact but hierarchical

Steps:

Think through the code in three short reasoning stages:

- (a) Identify block boundaries and types.
- (b) Detects relationships like contains, control_flow, or call.
- (c) Produce final JSON with exact line ranges.

Output only JSON as the final response.

Code with line numbers:

{lines}

TOT:

1. Progressive Refinement

Steps:

1. COARSE: Identify only functions, classes, major control (3-5 nodes max)
2. MEDIUM: Add assignments, calls within those blocks
3. FINE: Add relationships + line-precise boundaries

Build the graph progressively, keeping only valid relationships at each level.
Final output: Complete JSON from Level 3.

Code:

{lines}

2. Multi-path block

Steps:

Consider THREE block decomposition paths:

PATH A: Conservative (functions/classes only)

PATH B: Granular (assignments + calls)

PATH C: Hybrid (functions + key control)

Pick the BEST path. Output its complete JSON.

Code: {lines}

3. Confidence-Weighted Blocks

Steps:

Generate 3-5 candidate blocks. For each:

1. id, type, line range, code

2. Confidence: high/med/low

Use only HIGH confidence blocks + relationships. Output JSON.

Code: {lines}

AGENT 2

CHAIN OF THOUGHT:

1. Stepwise Diagnostic Chain

Step by step:

1. Parse the graph structure and list every node with its relations.
2. Identify each relation's role (contains, data_flow, control_flow, call).
3. Evaluate semantic validity based on the given rules (e.g., control_flow→assignment OK, but assignment→assignment BAD).
4. Detect missing data_flow paths to return or orphaned nodes.
5. For each faulty pattern, label the fault_type and explain using explicit relation paths (e.g., "b5→return missing").
6. Assign fault_score 0–10.
7. Output the strict JSON format provided.

2. Pattern-Driven Analytical Chain

Step by step:

For each semantic pattern (BROKEN_DEPENDENCY, FLOW_MISMATCH, INCOMPLETE_CONVERGENCE, etc.):

- Check all node pairs and their connecting edges.
- Test whether current relations satisfy or violate the pattern definition.
- When a violation is found, record fault_type, affected nodes, and the missing or incorrect path.
- Continue until all seven patterns are verified against the graph.
- Compile results into "faulty_nodes" and "fault_summary" fields.

3. Relation-Centric Fault Reasoning

For each node:

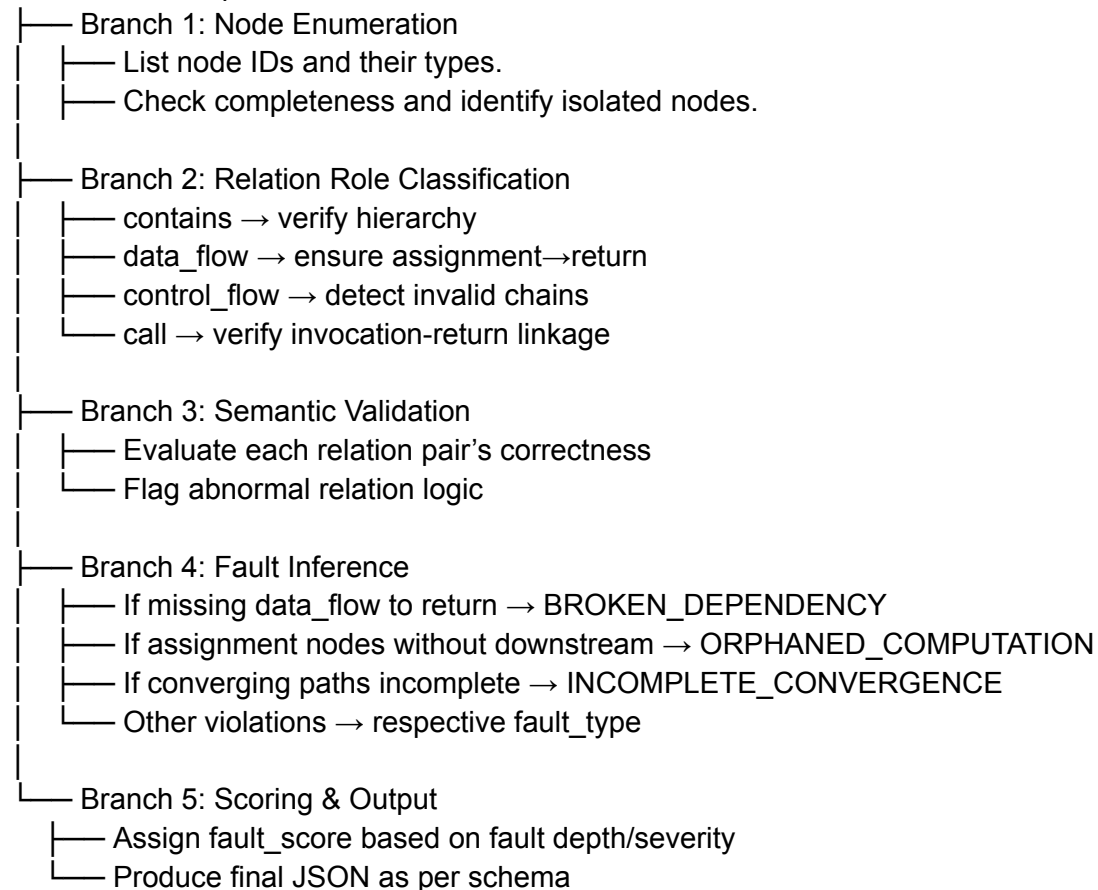
- List incoming and outgoing edges grouped by relation type.
- Analyze relation direction and compatibility based on semantic expectations.
- If a violation occurs, trace the minimal relation path causing it.
- Choose fault_type from the seven definitions, give a short reasoning phrase referencing node IDs and edge labels.
- Assign fault_score proportionally to severity.
- Confirm final JSON matches schema exactly.

TREE OF THOUGHT:

1. Stepwise Diagnostic Chain

Tree Expansion:

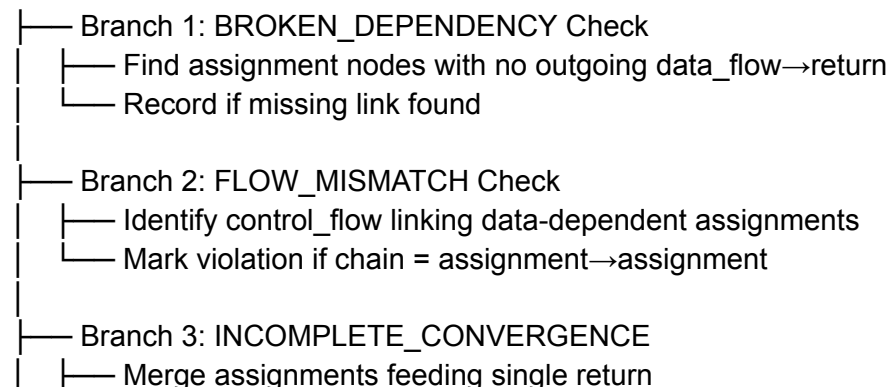
Root: Parse Graph

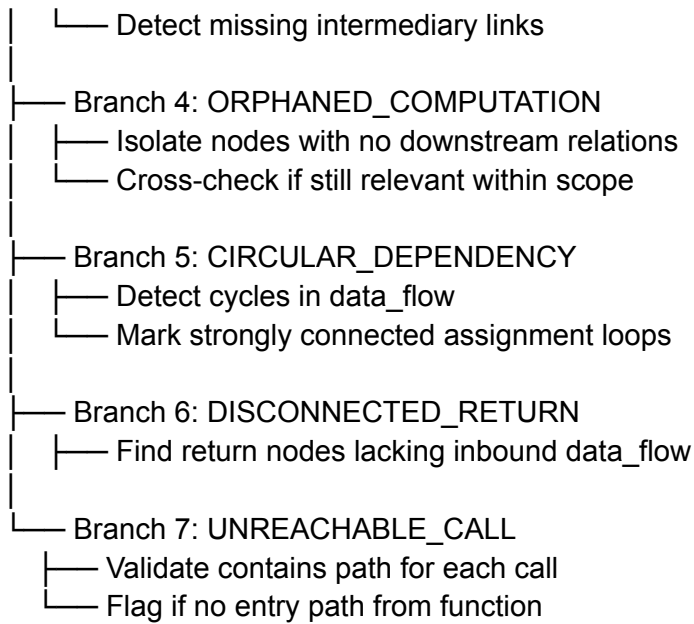


2. Pattern-Driven Analytical Chain

Tree Expansion:

Root: Scan for Semantic Patterns

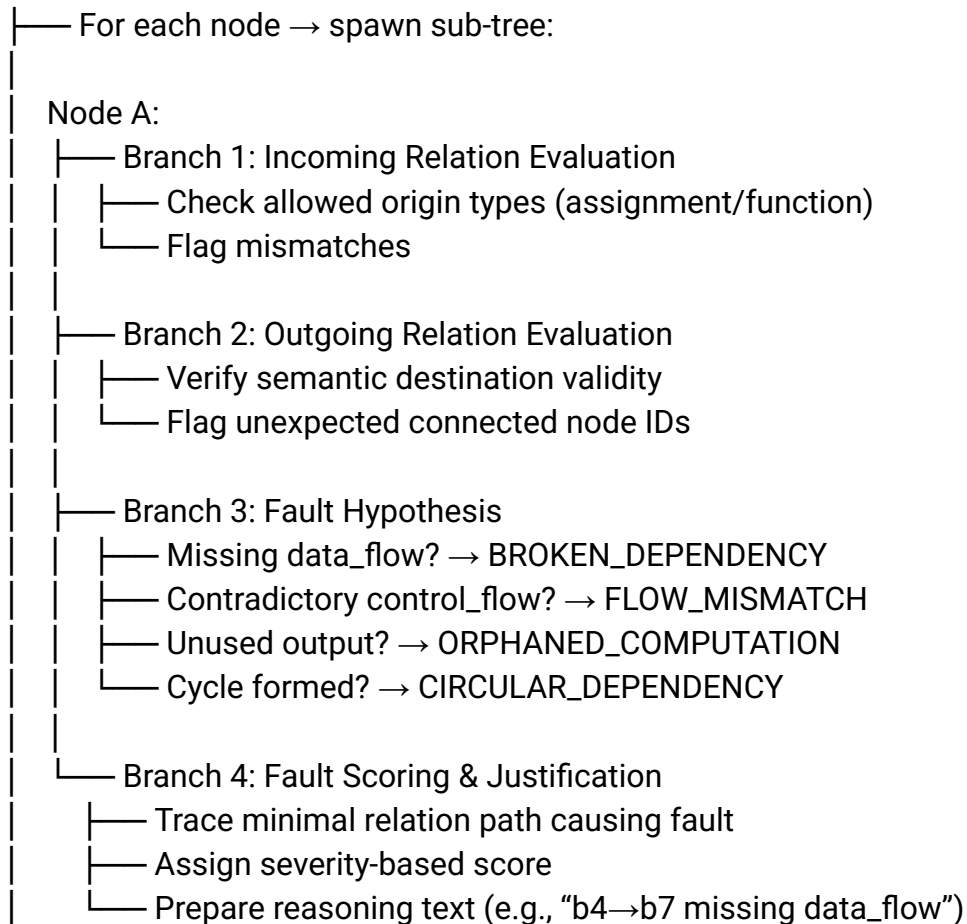


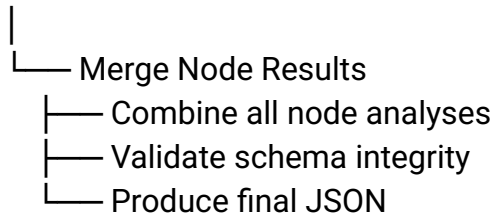


3. Relation-Centric Fault Reasoning

Tree Expansion:

Root: Per-Node Analysis





AGENT 3

COT:

1. Structured Reasoning Per Node

CODE RECTIFICATION AGENT – CHAIN OF THOUGHT MODE

Your job: Fix structural dependency errors using step-by-step reasoning.

=== STRICT PROCESS FOR EACH FAULTY NODE ===

1. INSPECT: List all incoming edges and outgoing edges. Identify producers and consumers.
2. DIAGNOSE: Explain exactly why this violates the fault_type rule.
3. PROPOSE: Describe the minimal fix (add edge, change label, or add temp node).
4. VERIFY: Confirm the fix preserves all other nodes/edges and fixes the fault.
5. APPLY: Integrate ONLY this fix into the full graph.

=== OUTPUT FORMAT ===

```
{
  "file_name": "{file_name}",
  "rectified_nodes": [
    {
      "node_id": "b4",
      "fault_type": "BROKEN_DEPENDENCY",
      "reasoning": [
        "1. INSPECT: In: control_flow b3→b4. Out: none.",
        "2. DIAGNOSE: b3 produces value used by b4, but no data_flow exists.",
        "3. PROPOSE: Add data_flow b3→b4.",
        "4. VERIFY: No cycles created, b4 now receives data.",
        "5. APPLY: Added edge."
      ],
      "fix_description": "Added data_flow b3→b4",
      "confidence": 9
    }
  ],
  "full_rectified_graph": "digraph G { ... COMPLETE FIXED GRAPH ... }",
```

```
"summary": {"total_fixes": 1, "edge_additions": 1}
}
```

JSON ONLY. Think in 'reasoning' array first, then output final graph.

2. Concise + Verify

AGENT3 FULL GRAPH REPAIR – OPTIMIZED COT

temperature=0.0 | JSON mode

PER-FAULTY-NODE STEPS (write these verbatim in output):

1. [NODE_ID]: In=[list edges], Out=[list edges]
2. Fault reason: [1 sentence]
3. Minimal fix: [edge add/change]
4. Post-fix check: [valid? cycles? all nodes?]
5. Confidence: [0-10]

Then output COMPLETE rectified graph.

FORMAT:

```
{
  "steps": ["1. b4: In=[b3], Out=[], ..."],
  "full_rectified_graph": "digraph G { ... }"
}
```

TOT:

1. Candidate Fixes + Selection

CODE RECTIFICATION AGENT – TREE OF THOUGHT MODE

For each faulty node: Generate 2 candidate fixes, evaluate, select best.

=== TREE SEARCH PROCESS ===

1. Generate 2 alternative minimal fixes.
2. Evaluate each: pros/cons, confidence, minimality score (0-10).
3. Select highest-scoring fix.
4. Apply ONLY the selected fix to graph.

=== OUTPUT FORMAT ===

```

{
  "file_name": "{file_name}",
  "rectified_nodes": [
    {
      "node_id": "b4",
      "fault_type": "CIRCULAR_DEPENDENCY",
      "candidates": [
        {
          "thought": "Add temp node t1 between b4→b5",
          "patch": "b4→t1 [data_flow]; t1→b5 [data_flow]",
          "pros": "Breaks cycle cleanly",
          "cons": "Adds 1 node",
          "score": 8
        },
        {
          "thought": "Relabel b5→b4 to control_flow",
          "patch": "b5→b4 [label=\"control_flow\"]",
          "pros": "No new nodes",
          "cons": "May break data dep",
          "score": 6
        }
      ],
      "selected": 0,
      "fix_description": "Added temp node t1 to break cycle",
      "confidence": 8
    }
  ],
  "full_rectified_graph": "digraph G { ... }",
  "summary": {"best_candidates_selected": 1}
}

```

JSON ONLY. Never apply non-selected fixes.

2. Multi-Node Coordination

AGENT3 – GLOBAL TREE OF THOUGHT

When fixes interact across nodes:

LEVEL 1: Fix each node independently (generate 2 options each)
LEVEL 2: Check interactions (new cycles? broken deps?)
LEVEL 3: Merge best compatible fixes

OUTPUT:

```
{
  "independent_fixes": [{"node_id": "b4", "options": [{"fix": "...", "local_score": 8}, ...]}],
  "merged_fix": "Final compatible set",
  "full_rectified_graph": "digraph G { ... }",
  "coordination_score": 9
}
```

JSON ONLY. Resolve conflicts at LEVEL 3.

AGENT 4

COT:

1.Line-by-Line Mapping

AGENT4 – STRUCTURED CHAIN OF THOUGHT EDITING

You reconstruct source code from graph fixes. CRITICAL: Preserve ALL original lines.

=== EXACT 5-STEP PROCESS (document each in output) ===

1. SCAN: Read original code line-by-line, note line numbers and node_ids referenced.
2. MAP: For each rectified_node, find EXACT matching line(s) in original code.
3. FIX: Apply ONLY the described fix to those line(s). Copy-paste surrounding code unchanged.
4. VERIFY: Confirm unchanged lines identical to original. All rectified_nodes fixed.
5. OUTPUT: Complete Python file with ONLY surgical changes.

=== JSON OUTPUT FIRST (reasoning only), THEN CODE ===

```
{
  "mapping": [
    "1. SCAN: 45 lines total. Nodes b4,b7 on lines 12,18.",
    "2. MAP: b4 fix → line 12 'x = df.head()', b7 → line 18.",
    "3. FIX: line 12 → 'x = df.head(10)' (added convergence).",
    "4. VERIFY: Lines 1-11,13-17,19-45 unchanged.",
    "5. READY: 3 lines changed, 42 preserved."
  ],

```

```
"changes_made": 3,  
"lines_preserved": 42  
}
```

/* NOW COMPLETE PYTHON CODE WITH ONLY VERIFIED FIXES */

```
=== ORIGINAL CODE ===  
{original_code}
```

```
=== ONLY FIX THESE NODES ===  
{rectified_nodes}
```

```
=== GRAPH CONTEXT ===  
{rectified_graph}
```

JSON reasoning first, then Python code. NO OTHER TEXT.

TOT:

1. Multi-Candidate Line Mapping Tree

AGENT4 – LINE MAPPING TREE OF THOUGHT

Use LINEAR CoT for simple nodes. Use TREE SEARCH for ambiguous mappings.

```
=== WHEN TO USE TREE (triggers) ===
```

- Node_id appears in 2+ code lines
- fix_description unclear (e.g. "add data_flow" vs specific var)
- Multiple possible edit locations

```
=== TREE SEARCH PER AMBIGUOUS NODE ===
```

```
|— MAP: Find all candidate lines containing node_id  
|— BRANCH 1: Edit option A → show before/after → verify graph fix applied?  
|— BRANCH 2: Edit option B → show before/after → verify graph fix applied?  
|— BRANCH 3: Edit option C (if needed) → verify  
|— SELECT: Highest verification score (preserves most code, matches fix)
```

```
=== OUTPUT FORMAT ===
```

```
{  
  "linear_mappings": [  
    "Node b4 → line 12 (simple, no tree needed)"  
  ],  
  "tree_mappings": [  
    {
```

```

"node_id": "b7",
"candidates": [
  {
    "line": 18,
    "before": "return x",
    "after": "return x,y",
    "matches_fix": "Yes: adds convergence y",
    "preserves_indent": "Yes",
    "score": 9
  },
  {
    "line": 20,
    "before": "result = x",
    "after": "result = x,y",
    "matches_fix": "Partial: wrong location",
    "score": 4
  }
],
"selected_line": 18,
"selected_edit": "return x → return x,y"
}
],
"total_trees": 1,
"confidence": 9
}

```

/* COMPLETE PYTHON CODE USING SELECTED EDITS ONLY */

{original_code}

{rectified_nodes}

{rectified_graph}

JSON trees first. Apply ONLY selected edits. Preserve ALL other lines.